

Failure (force the parser to actually fail) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `structured_output_practice.py` (Failure section): you need to know, before it happens in production, exactly what exception type a mismatched structured-output reply raises. Forcing it here, on purpose, once, means the Build Task's "handle a bad structured-output reply gracefully" requirement is catching an error you've actually seen fire, not a guess. **If not:** the first real failure would happen in front of a real user, with your code catching the wrong exception type or not catching it at all.

Basic Version

```
# structured_output_practice.py – Failure section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Rating(BaseModel):
    score: int

structured_model = ChatOpenAI().with_structured_output(Rating)

try:
    structured_model.invoke("Describe your favorite color, no numbers at all.")
except Exception as e:
    print("Error type:", type(e))
    print("Error message:", e)
```

This version works and shows you the failure. Catching the broad `Exception` finds it, but doesn't tell your future code exactly which class to catch on purpose — you still have to go read the printed type by hand.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# structured_output_practice.py – Failure section
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI

class Rating(BaseModel):
    score: int

def get_rating_chain() -> ChatOpenAI:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.with_structured_output(Rating)

def main() -> None:
    structured_model = get_rating_chain()

    try:
        prompt = "Describe your favorite color, no numbers at all."
        structured_model.invoke(prompt)
    except ValidationError as e:
        # why: catching the specific class means real code can react
        # differently to "didn't match my shape" than to anything else.
        print(f"Caught a ValidationError (expected): {e}")
    except Exception as e:
        print(
            f"Caught a different error than expected: {type(e).__name__}: {e}"
        )
        print(f"Full class chain: {type(e).__mro__}")

if __name__ == "__main__":
    main()
```

Difference from Basic: Basic catches everything with one broad `except Exception`. Intermediate catches the *specific* class (`ValidationError`) it expects, with a broader fallback that also prints the full class chain — this is exactly the "check the real type before assuming" habit Doc01's Core Concepts describes, applied here for real.

Which one should you actually write? Intermediate's specific-catch-plus-fallback is what this document's Build Task needs — it handles a bad structured-output reply gracefully, which is all the requirement asks for. A caller-friendly wrapper (returning `Rating | None`, logging and re-raising only genuinely unexpected error types) is worth writing once this call is made from inside a real feature rather than a debugging script; a retry with a clarified prompt is worth adding on top of that once you've confirmed a second attempt genuinely has a shot at succeeding. Neither is required here.